Hydra: Harnessing Expert Popularity for Efficient Mixture-of-Expert Inference on Chiplet System

Siqi He*†, Haozhe Zhu*†, Jiapei Zheng†, Lizhou Wu†, Bo Jiao†, Qi Liu†, Xiaoyang Zeng†, and Chixiao Chen†‡
† *State Key Laboratory of Integrated Chips and Systems (SKLICS), Fudan University, Shanghai, China*
‡ *Shaoxin Laboratory, Shaoxing, Zhejiang, China*
E-mail: zhuhz@fudan.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133023

***Abstract*—The rapid growth of model sizes in advanced artificial intelligence algorithms, particularly in Transformer-based large language models (LLMs), has led to significant computational overhead. Mixture-of-Expert (MoE) models offer a solution through their sparsely gating mechanism but introduce new challenges of extensive all-to-all communication and model computational inefficiencies. This paper presents Hydra, a software/hardware co-design aimed at accelerating MoE inference on chiplet-based architectures. In software, Hydra employs a popularity-aware expert mapping strategy to optimize inter-chiplet communication. In hardware, it incorporates Content Addressable Memory (CAM) to eliminate expensive explicit token (un)-permutation based on sparse matrix multiplications and a redundant-calculation-skipping softmax engine to bypass unnecessary division and exponential operations. Evaluated in 22 nm technology, Hydra achieves latency reductions of 14.2× and 3.5× and power reductions of 169.1× and 18.9× over GPU and state-of-the-art MoE accelerator, respectively, thereby offering a scalable and efficient solution for MoE model deployment.**

***Index Terms*—Mixture-of-expert model, chiplet, large language model**

I. Introduction

Transformer-based large language models (LLMs) have demonstrated exceptional performance in natural language processing tasks [1]–[3]. However, their advancements are primarily driven by the growing model sizes, which hinder efficient deployment on resource-constrained devices. Recently, Mixture-of-Expert (MoE) models have emerged as an effective solution to mitigate the computational overheads of LLMs [4]–[6]. As illustrated in Fig. 1, in an MoE model, the standard feed-forward network (FFN) layer in Transformers is replaced by an MoE FFN layer comprising multiple sparsely activated experts controlled by a gating network. This sparse gating mechanism enables further model expansion with only a slight increase in computational cost.

Despite their advantages, accommodating the entire MoE model on a single chip remains impractical due to limited on-chip memory. To overcome the cost and area limitations of monolithic chips, multi-chiplet modules (MCM) have emerged as a promising approach for building energy-efficient accelerators [7], [8]. By connecting multiple smaller chiplets through a silicon interposer or organic substrate, MCMs significantly expand system capacity [9], [10]. Additionally, their flexible interconnect supports various parallelism strategies commonly used in popular inference frameworks [11], especially model parallelism [6] and expert parallelism [12], to improve MoE inference efficiency [13]–[16].

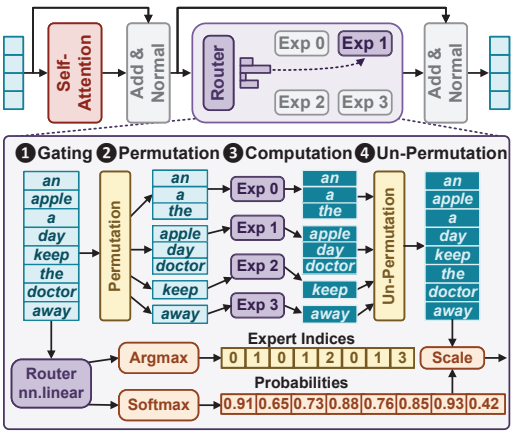

Fig. 1. Illustration of Mixture-of-Expert (MoE) mechanism in Transformer-based models.

However, deploying MoE models on chiplet-based architectures faces several challenges. First, as expert parallelism places different experts on different chiplets and executes them in parallel, extensive token dispatch and gathering operations across chiplets become necessary, resulting in extensive all-to-all communication. Previous studies have shown that this communication pattern can lead to network contention and congestion, limiting bandwidth utilization and ultimately becoming a bottleneck during inference [17]–[19]. Second, key operations in MoE layers remain computationally inefficient, including *permutation*, *un-permutation*, and *gating*. The (un)-permutation stage relies heavily on sparse matrix multiplications for token reordering, resulting in computational complexity of $O(S^2M)$, where S denotes the token length and M is the model hidden dimension. Moreover, the softmax function in the gating stage requires time-intensive exponential and division computations, incurring significant latency.

Prior approaches, including Pre-gated MoE [20] and FLAME [21], have focused on reducing the memory footprint of MoE models with an optimized expert prefetching strategy. However, these methods do not effectively address the all-to-all communication overhead or the intrinsic computational inefficiencies in MoE layers. To address these challenges, we propose Hydra, a hardware-software co-design. To our

* Siqi He and Haozhe Zhu are co-first authors. Haozhe Zhu is the corresponding author.

This work was supported by the National Natural Science Foundation of China (NSFC) under Grants 62304047 and 62322404.

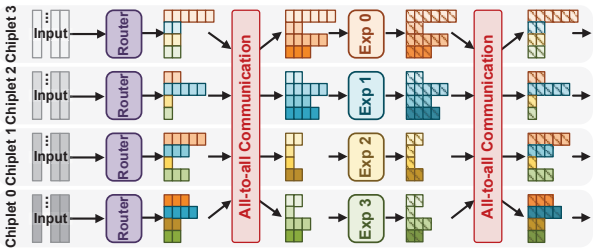

Fig. 2. MoE layer execution sequence, highlighting the all-to-all communication in an MCM, with the skewed routing decision favoring E_0 and E_1.

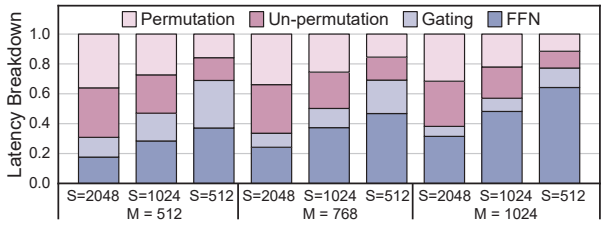

Fig. 3. Runtime analysis of four stages in an MoE layer (communication is excluded) for varying token lengths (S) and model hidden dimensions (M).

knowledge, Hydra is the first chiplet-based system specially designed for MoE deployment. The contributions of this work are as follows:

- We implement a popularity-aware dynamic expert mapping strategy derived from an analysis of conditional probabilities for expert selection across layers, significantly cutting all-to-all communication latency.
- We utilize CAM for parallel searching to establish a mapping between tokens and experts, eliminating the overhead of explicit (un)-permutation operations.
- We design a softmax engine with hybrid token/expert-wise calculation skipping, effectively removing redundant division and exponent operations in the gating stage.
- We develop the Hydra accelerator with 4×4 chiplets in 22 nm CMOS technology, achieving a 14.2× latency reduction and 169.1× power savings over the NVIDIA RTX 3090 GPU, positioning Hydra as a highly efficient solution for MoE-based LLM deployment.

II. Background and Motivation

A. MoE Execution in Chiplet-based System

As shown in Fig. 1, an MoE layer consists of four key steps: ❶ ***Gating***, where a gating network allocates input tokens to experts. Here, a linear layer first determines experts-token relevance, followed by softmax computation for probability assignment. An argmax unit then selects the target expert for each token and assigns the corresponding probability as the gate value to weight the output. Softmax computation, due to its division and exponential operations, dominates the computational cost of this stage. ❷ ***Permutation***, which groups tokens by their assigned experts, leveraging the *einsum* operator for sparse matrix multiplication $(S \times S) \times (S \times M)$, with a computational complexity of $O(S^2M)$. ❸ ***Computation***, where each expert processes their allocated tokens. ❹ ***Un-permutation***, which restores tokens to their original order, acting as the reverse of permutation stage. Together, these steps define MoE layer execution on a single device.

For multi-chiplet systems, MoE model deployment involves various parallelism strategies to improve throughput and reduce memory footprint, necessitating inter-chiplet communication. For attention layers, ***model parallelism*** partitions weight tensors across different chiplets, with all-reduce operations performed within model-parallel groups to aggregate results. For MoE layers, ***expert parallelism*** places different experts on distinct chiplets, reducing memory requirements per chiplet while enabling parallel execution. However, as each chiplet holds only a subset of experts, tokens assigned to experts on other chiplets must be dispatched to the appropriate chiplet and gathered back after step ❸ , resulting in two rounds of all-to-all communication. Fig. 2 illustrates this process, showing tokens of the same color aggregated onto their designated chiplets and subsequently retrieved.

B. Challenges and Motivations

1) All-to-all Communication: This communication pattern has been identified as a major bottleneck, contributing up to 74.9% of the total runtime of an MoE layer [17]. Although previous studies have introduced device-limited or topology-aware routing mechanisms along with token-dropping techniques to reduce communication costs and limit expert capacity, they risk degrading model performance and introducing additional training overhead [4], [22], [23].

As prior works suggest, the routing decisions in MoE models tend to be highly skewed due to the specialized knowledge domain of each expert [17], [19]. Usually, only a small subset of experts, such as E_0 and E_1 in Fig. 2, are heavily utilized. While these skewed routing decisions overload certain chiplets with the majority of tokens and lead to imbalanced data traffic that intensifies the all-to-all bottleneck, they also introduce correlation in expert choices across consecutive MoE layers, according to [18], [20]. However, this insight has not been fully explored for all-to-all communication optimization. ***By leveraging this intrinsic inter-layer correlation as a predictive basis for expert popularity on each chiplet***, we propose a dynamic expert mapping strategy to improve bandwidth utilization, as detailed in Section III.

2) Inefficient Computation: To identify computational bottlenecks in MoE inference, we analyze the runtime breakdown of the four key stages across different token lengths and model hidden dimensions. As shown in Fig. 3, excluding inter-chiplet communication, the (un)-permutation and gating stages together account for 62.7% of the runtime of an MoE layer on average, highlighting opportunities for further optimization. Although prior work has suggested using data replication for equivalent layout transformation [11], the complexity remains $O(S^2M/E)$, where E is the number of experts. Meanwhile, while optimizations for operators like argmax have been proposed [24], softmax remains unoptimized.

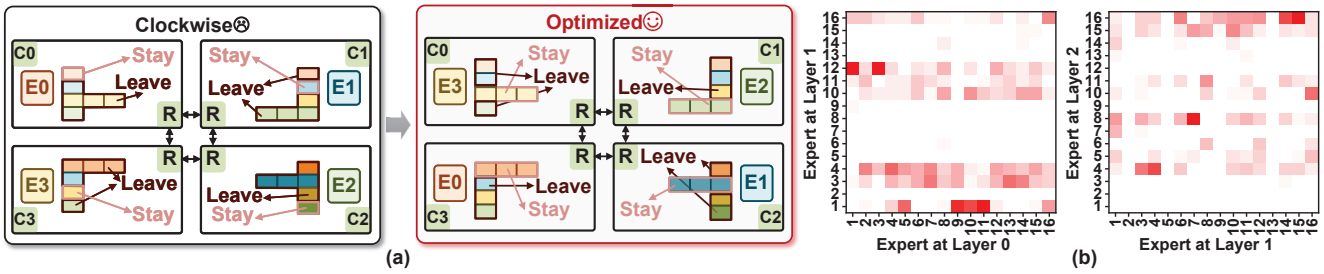

Fig. 4. (a) A 2×2-chiplet MCM example from a straightforward clockwise mapping scheme without considering expert popularity to an optimized mapping scheme that accounts for expert popularity across chiplets. (b) Heatmap of conditional probabilities for the Switch-base-16 model in the first two MoE layers.

To alleviate the computational burden, this paper introduces two key insights. First, ***a token-to-expert mapping established by parallel search*** can eliminate the need for sparse matrix multiplication. Second, the ***parallelism strategy combined with MoE's sparse gating mechanism introduces redundant computations*** in softmax operation. Based on these insights, we develop a CAM-based permutation engine for efficient parallel search and a softmax engine with hybrid token- and expert-wise calculation skipping to minimize inference latency. Further details are discussed in Section IV.

III. Software Optimization

To mitigate the all-to-all communication bottleneck in MoE inference, this section presents a predicted-popularity-aware expert mapping strategy, which is aimed at optimizing bandwidth usage and minimizing communication latency.

A. Expert Popularity Prediction

As illustrated in Fig. 4(a), for a 2×2 chiplet group with experts 0-3 arranged clockwise, each chiplet needs to send five tokens. However, with an optimized expert mapping scheme, each chiplet only sends three tokens, as more tokens remain on the original chiplet where their target experts reside. Thus, by considering the popularity of experts on each chiplet, an optimal mapping can be derived to either reduce inter-chiplet data transfers or minimize transmission hops, significantly lowering communication costs.

However, as routing decisions in MoE layers are dynamically determined at runtime, adjusting expert mapping based on the real-time gating results hinders time-overlapped weight loading, ultimately extending overall execution time. Therefore, predicting expert popularity in advance is essential for early expert mapping and prefetching. To this end, we track the routing decisions of 200,000 input tokens from the C4 dataset [25] as they traverse consecutive MoE layers and analyze the conditional probabilities of expert selection across various MoE models. Fig. 4(b) shows a heatmap of conditional probabilities for the Switch-base-16 model, revealing that expert selection across consecutive MoE layers is correlated. Therefore, expert popularity $\tilde{P}_c(E_{n,i})$ can be predicted as:

$$\tilde{P}_c(E_{n,i}) = \sum_{j=0}^{E-1} P_c(E_{n-1,j}) \times P(E_{n,i}|E_{n-1,j}) \quad (1)$$

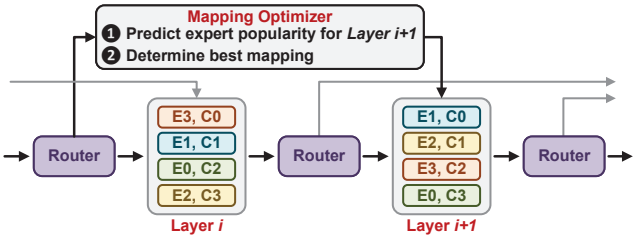

Fig. 5. Illustration of popularity-aware expert mapping.

In this equation, $P_c(E_{n-1,j})$ represents the real-time frequency of tokens assigned to expert j in layer $n-1$ for chiplet c, while $P(E_{n,i}|E_{n-1,j})$ represents the prior conditional probability of selecting expert i in layer n when expert j was selected in the previous layer $n-1$. This conditional probability-based prediction method enables the optimization of expert mapping.

B. Expert Mapping Algorithm

Building upon the expert popularity predictions presented in Section III-A, we develop an optimized expert mapping strategy, as shown in Fig. 5. Based on the prior conditional probability and the real-time routing decision in layer i, the expert mapping for layer $i+1$ can be determined. We first introduce a communication cost model tailored for multi-chiplet architectures, converting the problem into the following optimization task:

Given the expert popularity prediction $\tilde{P}_c(E_{n,i})$, the objective is to determine an optimal mapping between experts $\{E_{n,0}, E_{n,1}, ..., E_{n,E-1}\}$ and chiplets $\{C_0, C_1, ..., C_{C-1}\}$, where $n \in \{0, 1, ..., L-1\}$, $i \in \{0, 1, ..., E-1\}$ and $c \in \{0, 1, ..., C-1\}$. The mapping should minimize the following cost function:

$$\sum_{n=0}^{L-1} \sum_{c=0}^{C-1} \sum_{i=0}^{E-1} h(i,c) \times \tilde{P}_c(E_{n,i}) \quad (2)$$

Here, L and C represent the number of MoE layers and chiplets, respectively, while $h(i,c)$ quantifies the communication cost between the chiplet hosting expert i and chiplet c. This cost is approximated as the sum of the minimum horizontal and vertical hops in the interconnect topology.

To solve the above combinatorial optimization problem, we employ a simulated annealing algorithm, as detailed in Algorithm 1. The algorithm is deployed on the mapping solver in Hydra chiplet. During inference for layer n, the external memory interface retrieves the experts for layer $n+1$

based on the pre-computed mapping scheme from the solver. This approach mitigates all-to-all communication overhead and overlaps computation with memory access, effectively reducing latency and improving overall system throughput.

Algorithm 1 Expert Mapping using Simulated Annealing

```
mapping ← AssignExpertsToChipletsRandomly()
bestMapping ← mapping
bestCost ← CalculateCost(mapping)
Temp ← InitialTemperature
while not StoppingCondition do
    newMapping ← SwapTwoExperts(mapping)
    newCost ← CalculateCost(newMapping)
    delta ← newCost - bestCost
    if delta < 0 then
        bestMapping ← newMapping
        bestCost ← newCost
    end if
    Temp ← Temp × CoolingRate
end while
return bestMapping, bestCost
```

IV. Hardware Design

In this section, we present the detailed hardware implementation of Hydra, designed to enhance computational efficiency for MoE inference.

A. Overall Architecture

Fig. 6(a) illustrates the architecture of the Hydra multi-chiplet system, composed of a 2×2 Hydra chiplet array coupled with high-bandwidth memory. Similar to Simba [8], these Hydra chiplets are organized in a 2D grid and connected via ground reference signals (GRS), supporting 100 Gb/s bandwidth in each direction. This scalable configuration allows for expansion to larger chiplet arrays, enabling the deployment of MoE models with varying sizes and diverse computational demands. Fig. 6(b) details the design of a single Hydra chiplet, which comprises 16 processing element (PE) arrays, a global SRAM buffer, an auxiliary unit, a top controller, a mapping solver, a softmax engine, a CAM-based permutation engine, and die-to-die interconnects.

B. CAM-based Token Permutation Engine

The (un)-permutation stage employs sparse matrix multiplication to rearrange data. While crucial for all-to-all communication and FFN computation, (un)-permutation incurs significant computational overhead. Therefore, we introduce a CAM-based solution to establish a mapping between the token and expert to achieve equivalent data layout transformation with reduced complexity.

We adopt a conventional 9T NAND-type CAM design [26], which is implemented by incorporating additional NAND logic outside the basic 6T SRAM cell to facilitate the search operation. When all bits in a row of the CAM array match the search key, the match result remains at a high level. As this

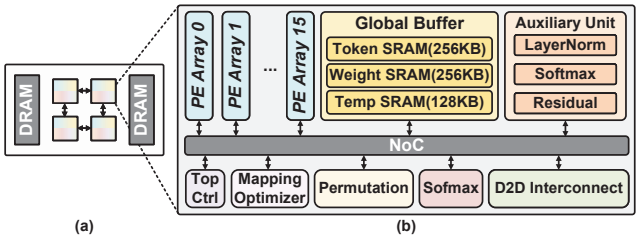

Fig. 6. (a) Overall architecture of Hydra MCM and (b) Hydra chiplet design.

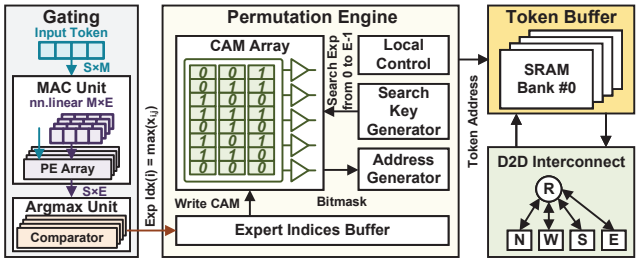

Fig. 7. CAM-based token permutation engine.

matching process is applied across all rows of the memory array, CAM enables rapid parallel searching.

The proposed CAM-based permutation engine is depicted in Fig. 7. Initially, the input tokens pass through a small linear layer. The output projection matrix is then passed to the *argmax* unit to identify the target expert, with corresponding expert indices stored in the CAM. The search key in the CAM is then sequentially set from 0 to $E-1$, with the match results forming a bitmask to indicate the positions of the relevant input tokens in memory. The address generator uses this bitmask to produce address indices for token retrieval, after which the tokens are fetched from the token buffer for participation in all-to-all communication. Following the completion of the FFN computation, the tokens are returned to the original chiplet through the interconnection network and stored back into the token buffer according to the bitmask.

The complexity of the parallel search operation in the CAM-based token permutation scheme is only $O(E)$. Given that CAM is far more energy-efficient in parallel search compared to conventional digital circuits, the overhead of this search is negligible relative to the extensive MAC operations in MoE inference. Thus, the token-to-expert mapping table is established at a minimal cost. Based on this mapping table, Hydra can complete the data layout transformation with just S data moves, achieving token permutation (or un-permutation) while eliminating the sparse matrix multiplication operations and minimizing intermediate results storage. This significantly reduces MoE inference latency and improves energy efficiency.

C. Redundant-Calculation-Skipping Softmax Engine

Fig. 8 illustrates the proposed softmax engine with hybrid token- and expert-wise calculation skipping. During probability calculation, exponential values for each element in the $S\times E$ output projection matrix of the linear layer are first computed and summed row-wise. The probability $S(x_{i,j})$ is subsequently derived through a division operation, which is implemented as a subtraction in the logarithmic domain.

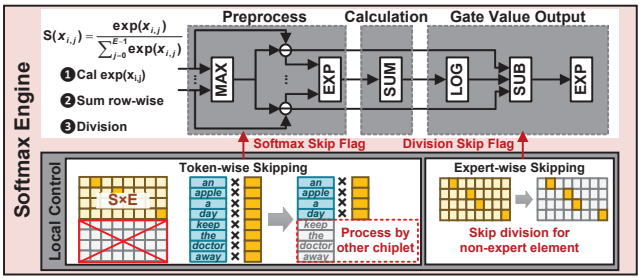

Fig. 8. Softmax engine with hybrid token-wise and expert-wise skipping.

The proposed token-wise skipping strategy targets redundant operations introduced by attention layers. Under model-parallelism, all-reduce communication occurs between chiplets that hold different weight tensor slices. As a result, input tokens across chiplets in the same model-parallel group are identical when entering the MoE layer. Thus, the local control unit instructs each chiplet to compute probabilities for only $1/M_P$ of tokens, with the remaining softmax operations assigned to other chiplet in the same group. Here, M_P is the degree of model parallelism. Expert-wise skipping strategy, on the other hand, aims at eliminating unnecessary division operations caused by the sparse gating mechanism. Once the *argmax* unit identifies the target expert, the local control unit skips the division operations for non-target experts, significantly reducing computational overhead.

The aforementioned token-wise skipping strategy is statically defined by the parallel configuration established during model deployment, while the expert-wise skipping strategy is dynamically determined by the routing decisions. As the proposed softmax skipping approach neither depends on data sparsity nor relies on approximate calculations, it offers an efficient, lossless computational optimization that eliminates redundancies arising from the inherent characteristics of the MoE algorithm and the parallelism framework.

V. Evaluation

A. Experimental Settings

Models and datasets. We evaluated the performance of Hydra using the Switch Transformer model [4], a state-of-the-art large-scale MoE model based on the encoder-decoder architecture. Specifically, we focused on switch-base-8 and switch-base-16 (with 8 and 16 experts per layer), as they are representative and highlight Hydra's scalability. The FP16 data type was employed. To verify that Hydra does not introduce any accuracy loss, we compared the negative log perplexity on a masked language modeling task against a GPU baseline using the C4 dataset.

Implementation. We evaluated a Hydra accelerator with a 4×4-chiplet configuration, using the Intel Xeon Gold 6242 and NVIDIA Geforce RTX 3090 as CPU and GPU benchmarks, respectively. The CAM array was implemented with custom circuits and simulated in Cadence Virtuoso. Power and area for inter-chip interconnects were scaled from previous 2.5D accelerator designs [8], consuming 2.4 pJ/bit. The rest of the digital modules were synthesized using Synopsys Design Compiler at 500 MHz under a 22 nm node. Table I shows the area and power breakdown. At 500 MHz, each Hydra chiplet occupies 10.49 mm^2 and consumes 934.65 mW of power.

TABLE I
Area and Power Breakdown of Hydra at 500MHz

Module	Area (mm^2)	Power (mW)
PE Arrays	5.77	643.98
Permutation Engine	0.01	0.86
Softmax Engine	0.14	19.36
Auxiliary Unit	0.26	35.43
Global Buffer	3.76	203.99
D2D Interface	0.06	4.38
Others	0.49	26.65
Total	**10.49**	**934.65**

Methodology. We used the s-tui tool [27] and NVIDIA System Management Interface to monitor the CPU and GPU, respectively. For the Hydra chip, we developed a behavioral-level simulator that was validated against the RTL implementation. At the chiplet level, we modified Noxim [28] to simulate inter-chiplet data transmission patterns, assessing both latency and power. For a thorough comparison, batch sizes of 1-4 were tested with various input-output token length combinations, where input lengths ranged from 64 to 1024 and output lengths from 16 to 256, reflecting typical user requests.

B. Performance

Inference latency and power consumption. Fig. 9(a) illustrates the normalized execution time of a single MoE layer across various input and output token lengths. The results show that Hydra achieves an average speedup of 14.2× over the GPU and 42.4× over the CPU. This significant performance improvement can be attributed to several key factors. First, Hydra optimizes expert mapping strategies, effectively reducing all-to-all communication latency. Additionally, Hydra leverages the CAM array and supports softmax skipping, minimizing the latency of the permutation operation and lowering the computational overhead of the gating stage. Fig. 9(b) presents the normalized power consumption of Hydra. The results demonstrate that Hydra achieves average power savings of 169.1× over the GPU and 357.5× over the CPU.

Multi-GPU implementation. Since multi-GPU systems also support expert parallelism, we deployed the MoE model on 4 to 8 NVIDIA A100 GPUs to compare the performance of Hydra with multi-GPU inference. Fig. 10 shows the normalized throughput and energy efficiency of Hydra under different batch sizes. The results indicate that, compared to implementations on 4 GPUs and 8 GPUs, Hydra achieves throughput improvements of 11.0× and 11.2×, respectively, along with energy efficiency improvements of 312.2× compared to 4 GPUs and 588.1× compared to 8 GPUs. The observed superiority of the 4 GPU implementation over the 8 GPU implementation may be attributed to increased communication overhead and resource contention, which can diminish the benefits of additional computational resources.

Ablation study. To evaluate the contribution of each proposed technique, we conduct ablation experiments. The work-

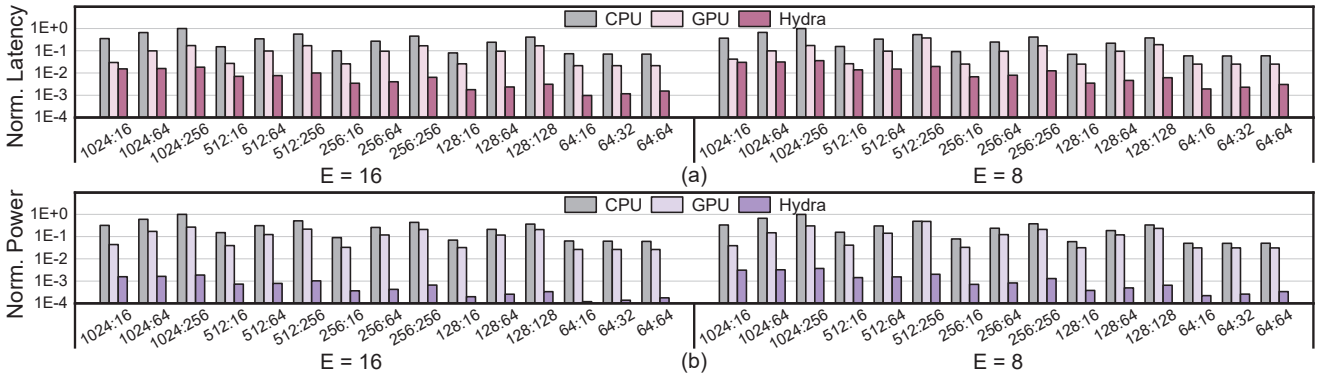

Fig. 9. (a) Inference latency and (b) power of Hydra compared to GPU and CPU. $E = 8$ refers to switch-base-8, and $E = 16$ refers to switch-base-16.

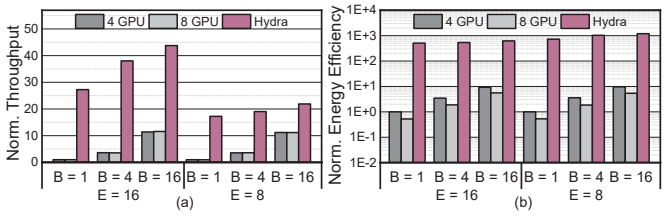

Fig. 10. Comparison with multi-GPU.

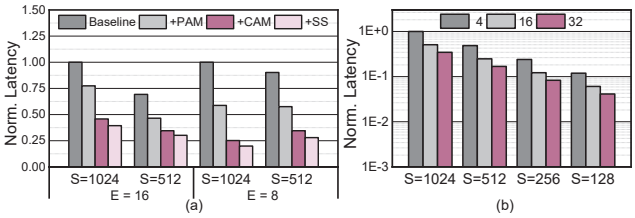

Fig. 12. (a) Ablation study. (b) Scalability evaluation.

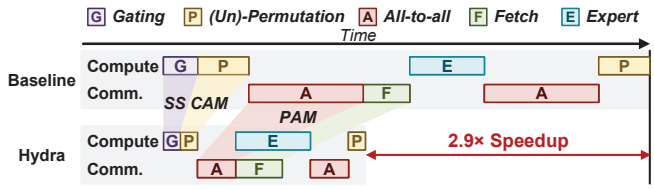

Fig. 11. Execution timeline. SS: softmax skipping; CAM: cam-based permutation; PAM: popularity-aware expert mapping.

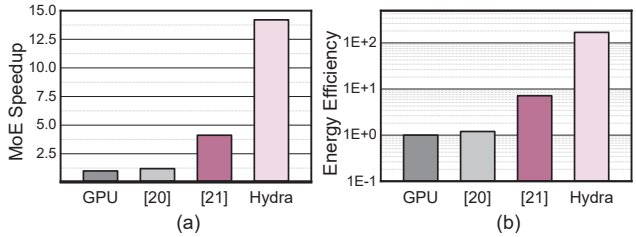

Fig. 13. (a) Inference speedup and (b) energy efficiency improvement of Hydra over other state-of-the-art works.

flow of Hydra with regard to the baseline is depicted in Fig. 11, and the results are displayed in Fig. 12(a). The optimized weight mapping strategy significantly reduces inference latency by 1.49× while introducing only a 1.2% increase in hardware overhead due to the mapping solver. Additionally, CAM-based permutation yields a 1.59× reduction by efficiently realizing token reordering, while the softmax-skipping technique further provides an additional 1.22× speedup. The above results demonstrate that each technology of Hydra has made significant contributions to inference acceleration.

Scalability. Fig. 12(b) demonstrates the performance scalability of Hydra when running the switch-base-16 model across different numbers of chiplets. When $C \leq E$, each chiplet holds E/C experts. Conversely, the MoE layer adopts a hybrid expert-model parallelism strategy, where each expert is distributed across C/E chiplets. While inter-chiplet communication costs generally limit the performance gains in chiplet-based systems, our expert mapping strategy efficiently optimizes bandwidth usage and minimizes all-to-all communication latency, thereby enabling Hydra to scale effectively.

Comparison with other accelerators. We compared Hydra to two state-of-the-art MoE acceleration schemes, Pre-gated MoE [20] and FLAME [21], both of which leverage memory offloading for single-device inference acceleration. Fig. 13 illustrates the speedup and energy efficiency results on the switch-base-8 model. Hydra achieves 14.2×, 11.8×, and 3.5× speedups over GPU-only, Pre-gated MoE, and FLAME, respectively, demonstrating the capability of a chiplet-based architecture in addressing MoE model memory requirements while alleviating latency bottlenecks from all-to-all communication through optimized expert mapping. Hydra also realizes energy efficiency improvements of 169.1×, 141.1×, and 18.9× over GPU-only, Pre-gated MoE, and FLAME, respectively.

VI. Conclusion

This work presents Hydra, a software/hardware co-design developed to address the communication and computational bottlenecks in MoE model deployment. By predicting expert popularity and dynamically adjusting expert mapping, Hydra significantly optimizes all-to-all communication. By incorporating a CAM array for (un)-permutation operations and eliminating redundant softmax calculations, Hydra further enhances computational efficiency. Hydra achieves significant reductions in latency and power, with average improvements of 14.2× and 169.1× over the NVIDIA RTX 3090 GPU, highlighting its effectiveness in accelerating MoE inference.

REFERENCES

[1] J. Achiam *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[3] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[4] W. Fedus *et al.*, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *JMLR*, vol. 23, no. 120, pp. 1–39, 2022.

[5] Y. Zhou *et al.*, "Mixture-of-experts with expert choice routing," in *NIPS*, vol. 35, 2022, pp. 7103–7114.

[6] C. Hwang *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *MLS*, vol. 5, 2023, pp. 269–287.

[7] S. Naffziger *et al.*, "2.2 amd chiplet architecture for high-performance server and desktop products," in *ISSCC*, 2020, pp. 44–45.

[8] Y. S. Shao *et al.*, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *MICRO*, 2019, pp. 14–27.

[9] P. Vivet *et al.*, "Intact: A 96-core processor with six chiplets 3d-stacked on an active interposer with distributed interconnects and integrated power management," *JSSC*, vol. 56, no. 1, pp. 79–97, 2021.

[10] F. Li *et al.*, "Gia: A reusable general interposer architecture for agile chiplet integration," in *ICCAD*, 2022, pp. 1–9.

[11] R. Y. Aminabadi *et al.*, "DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale," in *SC*, 2022, pp. 1–15.

[12] D. Lepikhin *et al.*, "Gshard: Scaling giant models with conditional computation and automatic sharding," in *ICLR*, 2020.

[13] J. Cai *et al.*, "Gemini: Mapping and architecture co-exploration for large-scale dnn chiplet accelerators," in *HPCA*, 2024, pp. 156–171.

[14] Y. Feng *et al.*, "Heterogeneous die-to-die interfaces: Enabling more flexible chiplet interconnection systems," in *MICRO*, 2023, pp. 930–943.

[15] Z. Tan *et al.*, "Cocco: Hardware-mapping co-exploration towards memory capacity-communication optimization," in *ASPLOS*, 2024, pp. 69–84.

[16] Y. Feng *et al.*, "Evaluating chiplet-based Large-Scale interconnection networks via Cycle-Accurate Packet-Parallel simulation," in *USENIX ATC*, 2024, pp. 731–747.

[17] J. Li, Y. Jiang, Y. Zhu, C. Wang, and H. Xu, "Accelerating distributed MoE training and inference with lina," in *USENIX ATC*, 2023, pp. 945–959.

[18] J. Yao *et al.*, "Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference," in *IPDPS*, 2024, pp. 915–925.

[19] J. He *et al.*, "Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models," in *PPoPP*, 2022, pp. 120–134.

[20] R. Hwang *et al.*, "Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *ISCA*, 2024, pp. 1018–1031.

[21] L. Xuanda *et al.*, "Flame: Fully leveraging moe sparsity for transformer on fpga," in *DAC*, 2024, pp. 1–6.

[22] A. Liu *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[23] C. Chen *et al.*, "Ta-moe: Topology-aware large scale mixture-of-expert training," in *NIPS*, vol. 35, 2022, pp. 22 173–22 186.

[24] X. Nie *et al.*, "Hetumoe: An efficient trillion-scale mixture-of-expert distributed training system," *arXiv preprint arXiv:2203.14685*, 2022.

[25] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *arXiv preprint arXiv:1910.10683*, 2019.

[26] K. Pagiamtzis *et al.*, "Content-addressable memory (cam) circuits and architectures: a tutorial and survey," *JSSC*, vol. 41, no. 3, pp. 712–727, 2006.

[27] A. Manuskin, "The stress terminal ui: s-tui." [Online]. Available: https://github.com/amanusk/s-tui, 2019.

[28] V. Catania *et al.*, "Noxim: An open, extensible and cycle-accurate network on chip simulator," in *ASAP*, 2015, pp. 162–163.